

PRD — API do Sistema de Gestão de Competições Esportivas

Autor: Pleno Backend Engineer **Objetivo deste documento:** especificar todos os endpoints da API para que o time de frontend construa mocks e comece o desenvolvimento em paralelo ao backend, e para que agentes de IA usados no desenvolvimento (Claude Code, Gemini CLI etc.) tenham contrato suficiente para gerar controllers, DTOs e testes sem ambiguidade.

Este é um documento **spec-driven**: cada Task abaixo é uma unidade de trabalho fechada, com contrato de API completo, que pode ser implementada, mockada e testada de forma independente. A ordem das Tasks reflete dependência de dados (ex: TASK-07 depende de Tournament existir, que depende de TASK-03 e TASK-02).

1. Visão geral

Dois públicos consomem essa API:

- **Staff** (autenticado, **Bearer <JWT>**): cadastra competições, edições, modalidades, times, atletas, monta torneios, registra partidas e eventos. Permissões são escopadas por edição via **EditionStaffRole** (exceto **isSuperAdmin**, que é global).
- **Público/Espectador** (sem autenticação): consulta torneios, partidas, placares e classificação — inclusive em tempo real via SSE.

Base URL: **/api/v1** **Formato:** JSON (**Content-Type: application/json**) em todos os endpoints, exceto o stream de eventos (**text/event-stream**).

2. Convenções gerais da API

2.1 Envelope de resposta

Todas as respostas de sucesso seguem este formato:

```
{  
  
  "data": { /* objeto ou array */ },
```

```
"meta": { /* presente apenas em listagens paginadas */ }  
}
```

Listagens paginadas (**meta**):

```
{  
  
  "data": [ /* ... */ ],  
  
  "meta": { "page": 1, "pageSize": 20, "total": 87, "totalPages": 5 }  
}
```

Paginação é via query params **?page=1&pageSize=20** (default **pageSize=20**, máximo **100**).

2.2 Formato de erro

```
{  
  
  "error": {  
  
    "code": "VALIDATION_ERROR",  
  
    "message": "O campo 'year' é obrigatório.",  
  
    "details": [{ "field": "year", "issue": "required" }]  
  
  }  
}
```

Códigos usados no MVP: **VALIDATION_ERROR** (400), **UNAUTHORIZED** (401), **FORBIDDEN** (403), **NOT_FOUND** (404), **CONFLICT** (409, ex: violação de **@@unique**), **INTERNAL_ERROR** (500).

2.3 Autenticação

Header **Authorization: Bearer <accessToken>** em todo endpoint marcado como **Staff** ou **Admin** abaixo. Tokens expiram em 15 min; renovação via refresh token (TASK-01).

2.4 Hierarquia de papéis e escopo de edição

O sistema tem três níveis de permissão, cada um contido no anterior:

Papel	Escopo	Responsabilidades-chave
SuperAdmin (<code>Staff.isSuperAdmin</code>)	Global	Mantém o sistema. Cria <code>Competition/CompetitionEdition</code> . É o único que promove alguém a <code>EDITION_ADMIN</code> .
EDITION_ADMIN	Uma <code>CompetitionEdition</code> inteira	Cadastra <code>Team/Athlete</code> (catálogo global). Cria <code>Discipline</code> e a associa à edição. Promove <code>DISCIPLINE_MANAGER</code> por modalidade. Herda todas as permissões de <code>DISCIPLINE_MANAGER</code> em qualquer modalidade dessa edição.
DISCIPLINE_MANAGER	Uma modalidade (<code>disciplineId</code>) dentro de uma edição	Seleciona quais times/atletas já cadastrados participam da sua modalidade, define o formato do torneio, configura duração de partida e critérios de desempate, inicia e atualiza partidas, registra os eventos da partida (gol, cartão, ponto).

Regra de autorização no backend: "o Staff tem `isSuperAdmin` OU tem um `EditionStaffRole` com papel igual-ou-superior ao exigido, no escopo correto (`editionId` e, quando aplicável, `disciplineId`)." Como `EDITION_ADMIN` herda `DISCIPLINE_MANAGER`, a checagem de "pode fazer ação de `DISCIPLINE_MANAGER`?" deve aceitar tanto um `EditionStaffRole` de `DISCIPLINE_MANAGER` naquela `disciplineId` quanto um de `EDITION_ADMIN` naquela `editionId` (com `disciplineId: null`).

Nota de design: `Discipline` é catálogo global (compartilhado entre eventos, ex: "Futsal" serve tanto para Jogos de Engenharia quanto Jogos de Informática), mas quem cria/associa é `EDITION_ADMIN`, que é escopado a uma edição. Isso é intencional pela simplicidade operacional — na prática um só evento roda por vez — mas fica registrado aqui como trade-off consciente, não acidental.

2.5 IDs

Todos os IDs são **cuid** (string). Exemplo de formato: "**clx1a2b3c0000ab12cd34ef56**".

3. Índice de Tasks

Task	Domínio	Depende de
TASK-01	Autenticação de Staff	—
TASK-02	Competitions & Editions	TASK-01
TASK-03	Disciplines (catálogo global)	TASK-01
TASK-04	Teams & Athletes (catálogo global)	TASK-01
TASK-05	Edition Roster (inscrição de atletas/times na edição)	TASK-02, 03, 04
TASK-06	Edition Staff Roles (permissões escopadas)	TASK-01, 02
TASK-07	Tournaments	TASK-02, 03
TASK-08	Phases & Groups	TASK-07
TASK-09	Tournament Entries	TASK-05, 07, 08
TASK-10	Matches	TASK-08, 09
TASK-11	Match Events	TASK-10
TASK-12	Real-time (SSE)	TASK-10, 11
TASK-13	Phase Standings (classificação)	TASK-08, 10
TASK-14	Audit Logs	todos acima
TASK-15	Rotas públicas agregadas (spectator)	TASK-07 a 13

TASK-01 — Autenticação de Staff

Modelo: **Staff**

Método	Rota	Auth
POST	<code>/auth/login</code>	Público
POST	<code>/auth/refresh</code>	Público (via cookie/refresh token)
POST	<code>/auth/logout</code>	Staff
GET	<code>/auth/me</code>	Staff

POST `/auth/login`

Request:

```
{ "email": "coordenador@ufpe.br", "password": "senha-forte" }
```

Response **200**:

```
{  
  "data": {  
    "accessToken": "eyJhbGciOi...",  
    "refreshToken": "eyJhbGciOi...",  
    "expiresIn": 900,  
    "staff": {  
      "id": "clx_staff_01",  
      "name": "Ana Coordenadora",  
      "email": "coordenador@ufpe.br",  
      "isSuperAdmin": false  
    }  
  }  
}
```

```
}  
  
}  
  
}
```

Erros: **401 UNAUTHORIZED** (credenciais inválidas).

GET /auth/me

Response **200**:

```
{  
  
  "data": {  
  
    "id": "clx_staff_01",  
  
    "name": "Ana Coordenadora",  
  
    "email": "coordenador@ufpe.br",  
  
    "isSuperAdmin": false,  
  
    "editionRoles": [  
  
      {  
  
        "editionId": "clx_edition_2026",  
  
        "editionName": "Jogos de Engenharia 2026",  
  
        "disciplineId": "clx_disc_futsal",  
  
        "disciplineName": "Futsal",  
  
        "role": "DISCIPLINE_MANAGER"  
  
      }  
  
    ]  
  
  }  
}
```

```
}  
  
}
```

Nota para o frontend: use `editionRoles` para decidir quais ações mostrar na UI (ex: esconder "criar torneio" se não houver `EDITION_ADMIN/DISCIPLINE_MANAGER` na edição atual). O backend também valida no servidor — isso é só para UX.

TASK-02 — Competitions & Editions

Modelos: `Competition`, `CompetitionEdition`

Método	Rota	Auth
GET	<code>/competitions</code>	Público
POST	<code>/competitions</code>	Admin (isSuperAdmin)
GET	<code>/competitions/:id</code>	Público
GET	<code>/competitions/:id/editions</code>	Público
POST	<code>/competitions/:id/editions</code>	Admin
GET	<code>/editions/:editionId</code>	Público
PATCH	<code>/editions/:editionId</code>	Admin ou <code>EDITION_ADMIN</code>
PATCH	<code>/editions/:editionId/status</code>	Admin ou <code>EDITION_ADMIN</code>

`POST /competitions`

// request

```
{ "name": "Jogos de Engenharia", "slug": "jogos-de-engenharia" }
```

`POST /competitions/:id/editions`

// request

```
{
```

```
"year": 2026,  
  
"name": "Jogos de Engenharia 2026",  
  
"startDate": "2026-10-12T00:00:00Z",  
  
"endDate": "2026-10-19T00:00:00Z"  
  
}
```

```
// response 201
```

```
{  
  
  "data": {  
  
    "id": "clx_edition_2026",  
  
    "competitionId": "clx_comp_01",  
  
    "year": 2026,  
  
    "name": "Jogos de Engenharia 2026",  
  
    "startDate": "2026-10-12T00:00:00Z",  
  
    "endDate": "2026-10-19T00:00:00Z",  
  
    "status": "PLANNING"  
  
  }  
  
}
```

Erros: 409 **CONFLICT** se já existir edição com esse (**competitionId**, **year**).

PATCH /editions/:editionId/status

```
// request
```

```
{ "status": "ONGOING" }
```


`status` ∈ PLANNING | ONGOING | FINISHED | ARCHIVED.

TASK-03 — Disciplines (catálogo global)

Modelos: `Discipline`, `EditionDiscipline`

Método	Rota	Auth
GET	<code>/disciplines</code>	Público
POST	<code>/disciplines</code>	<code>EDITION_ADMIN</code>
GET	<code>/editions/:editionId/disciplines</code>	Público
POST	<code>/editions/:editionId/disciplines</code>	<code>EDITION_ADMIN</code>
PATCH	<code>/editions/:editionId/disciplines/:id</code>	<code>EDITION_ADMIN</code> ou <code>DISCIPLINE_MANAGER</code> daquela modalidade
DELETE	<code>/editions/:editionId/disciplines/:disciplineId</code>	<code>EDITION_ADMIN</code>

`POST /disciplines`

```
{ "name": "Futsal", "slug": "futsal", "isIndividual": false, "description": null }
```

`POST /editions/:editionId/disciplines`

Associa uma modalidade do catálogo a essa edição, com config específica.

// request

```
{  
  
  "disciplineId": "clx_disc_volei",  
  
  "config": { "setsToWin": 3, "pointsPerSet": 25 }  
}
```

`config` é livre (JSONB) — o frontend deve tratá-lo como um formulário dinâmico por modalidade, não como schema fixo.

`EDITION_ADMIN` cria o vínculo `EditionDiscipline` (decide **quais** modalidades a edição vai ter). Depois de promovido, o `DISCIPLINE_MANAGER` daquela modalidade específica refina o `config` (ex: duração de partida) via:

```
// PATCH /editions/:editionId/disciplines/:id — request
```

```
{ "config": { "matchDurationMinutes": 40 } }
```

```
GET /editions/:editionId/disciplines
```

```
{  
  "data": [  
    {  
      "id": "clx_ed_disc_01",  
      "disciplineId": "clx_disc_futsal",  
      "disciplineName": "Futsal",  
      "isIndividual": false,  
      "config": { "matchDurationMinutes": 40 }  
    }  
  ]  
}
```

TASK-04 — Teams & Athletes (catálogo global)

Modelos: `Team`, `Athlete`

Método	Rota	Auth
GET	/teams?search=	Staff
POST	/teams	EDITION_ADMIN
GET	/teams/:id	Staff
GET	/athletes?search=	Staff
POST	/athletes	EDITION_ADMIN
GET	/athletes/:id	Staff
GET	/athletes/:id/history	Staff

Nota de autorização: o cadastro de **Team/Athlete** fica com **EDITION_ADMIN**, não **SuperAdmin** (evita gargalo de uma pessoa só) nem **DISCIPLINE_MANAGER** (que só *seleciona*, dentre os já cadastrados, quem participa da própria modalidade — ver TASK-05 e TASK-09). Isso reflete a separação: **EDITION_ADMIN** cuida do cadastro amplo da edição, **DISCIPLINE_MANAGER** cuida da curadoria por modalidade.

POST /athletes

// request

```
{
  "name": "João Pedro Silva",
  "document": "123.456.789-00",
  "birthDate": "2003-05-14",
  "email": "joao.silva@ufpe.br"
}
```

Erros: **409 CONFLICT** se **document** já cadastrado (é **@unique**).

GET /athletes/:id/history

Retorna todas as **EditionRoster** do atleta — histórico entre edições/eventos (o caso de uso que motivou o modelo global de **Athlete**).

```
{  
  "data": [  
    {  
      "editionName": "Jogos de Engenharia 2026",  
      "disciplineName": "Futsal",  
      "teamName": "CIn FC",  
      "jerseyNumber": 10,  
      "status": "ACTIVE"  
    },  
    {  
      "editionName": "Jogos de Informática 2027",  
      "disciplineName": "Xadrez",  
      "teamName": null,  
      "jerseyNumber": null,  
      "status": "ACTIVE"  
    }  
  ]  
}
```

TASK-05 — Edition Roster

Modelo: `EditionRoster`

Método	Rota	Auth
GET	/editions/:editionId/rosters?disciplineId=&teamId=	Público
POST	/editions/:editionId/rosters	DISCIPLINE_MANAGER/EDITION_ADMIN
PATCH	/editions/:editionId/rosters/:id	DISCIPLINE_MANAGER/EDITION_ADMIN
DELETE	/editions/:editionId/rosters/:id	EDITION_ADMIN

POST /editions/:editionId/rosters

// request

```
{
  "disciplineId": "clx_disc_futsal",
  "athleteId": "clx_athlete_09",
  "teamId": "clx_team_cinfc",
  "jerseyNumber": 10
}
```

`teamId` deve ser omitido/`null` quando `discipline.isIndividual === true`.

Erros: **409 CONFLICT** se o atleta já tiver inscrição nessa (`editionId`, `disciplineId`).

PATCH /editions/:editionId/rosters/:id

Usado para mudar `status` (ex: lesão, suspensão):

```
{ "status": "INJURED" }
```

`status` ∈ ACTIVE | INJURED | SUSPENDED | WITHDRAWN.

TASK-06 — Edition Staff Roles

Modelo: `EditionStaffRole`

Método	Rota	Auth
GET	<code>/editions/:editionId/staff-roles</code>	<code>EDITION_ADMIN</code>
POST	<code>/editions/:editionId/staff-roles</code>	Ver nota abaixo
DELETE	<code>/editions/:editionId/staff-roles/:id</code>	Ver nota abaixo

Nota de autorização: o backend valida por `role` no corpo da requisição — não é uma auth única para a rota inteira:

- `role: "EDITION_ADMIN"` → exige `isSuperAdmin`. É o único jeito de alguém virar `EDITION_ADMIN`.
- `role: "DISCIPLINE_MANAGER"` → exige `EDITION_ADMIN` naquela edição (ou `isSuperAdmin`).

Mesma regra vale para `DELETE`: remover um `EDITION_ADMIN` exige `isSuperAdmin`; remover um `DISCIPLINE_MANAGER` exige `EDITION_ADMIN`.

`POST /editions/:editionId/staff-roles`

// request

```
{  
  
  "staffId": "clx_staff_04",  
  
  "disciplineId": "clx_disc_futsal",  
  
  "role": "DISCIPLINE_MANAGER"  
}
```

`disciplineId: null` só é válido para `role: "EDITION_ADMIN"` (papel vale a edição inteira). Para `role: "DISCIPLINE_MANAGER"`, `disciplineId` é obrigatório. `role ∈ EDITION_ADMIN | DISCIPLINE_MANAGER`.

TASK-07 — Tournaments

Modelo: `Tournament`

Método	Rota	Auth
GET	<code>/editions/:editionId/tournaments?status=&disciplineId=</code>	Público
POST	<code>/editions/:editionId/tournaments</code>	<code>DISCIPLINE_MANAGER/EDITION_ADMIN</code>
GET	<code>/tournaments/:id</code>	Público
PATCH	<code>/tournaments/:id</code>	<code>DISCIPLINE_MANAGER/EDITION_ADMIN</code>
PATCH	<code>/tournaments/:id/status</code>	<code>DISCIPLINE_MANAGER/EDITION_ADMIN</code>

`POST /editions/:editionId/tournaments`

// request

```
{  
  
  "disciplineId": "clx_disc_futsal",  
  
  "name": "Futsal Masculino",  
  
  "format": "GROUP_KNOCKOUT"  
}
```

`format ∈ SINGLE_ELIMINATION | GROUP_KNOCKOUT | LEAGUE_KNOCKOUT | LEAGUE_ONLY | LEAGUE_LIMITED_KNOCKOUT`.

```
// response 201
```

```
{  
  
  "data": {  
  
    "id": "clx_tour_futsal_m",  
  
    "editionId": "clx_edition_2026",  
  
    "disciplineId": "clx_disc_futsal",  
  
    "name": "Futsal Masculino",  
  
    "format": "GROUP_KNOCKOUT",  
  
    "status": "DRAFT"  
  
  }  
}
```

```
PATCH /tournaments/:id/status
```

```
{ "status": "SCHEDULED" }
```

```
status ∈ DRAFT | SCHEDULED | ONGOING | FINISHED | CANCELLED.
```

Nota para o frontend: a criação de um Tournament **não** cria Phases automaticamente — isso é a TASK-08. Um Tournament em **DRAFT** sem Phases é um estado válido (staff ainda configurando).

TASK-08 — Phases & Groups

Modelos: **Phase**, **Group**, **GroupEntry**

Método	Rota	Auth
GET	/tournaments/:tournamentId/phases	Público

Método	Rota	Auth
POST	/tournaments/:tournamentId/phases	DISCIPLINE_MANAGER/EDITION_ADMIN
POST	/phases/:phaseId/groups	DISCIPLINE_MANAGER/EDITION_ADMIN
POST	/groups/:groupId/entries	DISCIPLINE_MANAGER/EDITION_ADMIN
DELETE	/groups/:groupId/entries/:entryId	DISCIPLINE_MANAGER/EDITION_ADMIN

POST /tournaments/:tournamentId/phases

// request — exemplo de fase de grupos

```
{
  "order": 1,
  "name": "Fase de Grupos",
  "type": "GROUP",
  "config": { "advanceCount": 2, "tiebreakers": ["points", "headToHead", "goalDiff"] }
}
```

// request — exemplo de mata-mata (2ª fase do mesmo torneio)

```
{
  "order": 2,
  "name": "Mata-mata",
  "type": "KNOCKOUT",
  "config": {}
}
```

`type` ∈ `GROUP` | `LEAGUE` | `KNOCKOUT`. `config` varia por `type` — ver comentários no schema; o frontend deve tratar como formulário dinâmico.

`POST /phases/:phaseId/groups`

`{ "name": "Grupo A" }`

`POST /groups/:groupId/entries`

`{ "entryId": "clx_entry_cinfc" }`

(`entryId` referencia um `TournamentEntry` já criado — ver TASK-09. A ordem correta de setup é: criar `TournamentEntry` primeiro, depois alocar em grupos.)

TASK-09 — Tournament Entries

Modelo: `TournamentEntry`

Método	Rota	Auth
GET	<code>/tournaments/:tournamentId/entries</code>	Público
POST	<code>/tournaments/:tournamentId/entries</code>	<code>DISCIPLINE_MANAGER/EDITION_ADMIN</code>
DELETE	<code>/tournaments/:tournamentId/entries/:id</code>	<code>DISCIPLINE_MANAGER/EDITION_ADMIN</code>

`POST /tournaments/:tournamentId/entries`

Coletivo:

`{ "teamId": "clx_team_cinfc", "seed": 1 }`

Individual (modalidade `isIndividual: true`):

`{ "athleteId": "clx_athlete_09", "seed": null }`

Exatamente um de `teamId/athleteId` deve ser enviado — o backend valida contra `Discipline.isIndividual` do torneio.

Erros: `409 CONFLICT` se o time/atleta já estiver inscrito nesse torneio.

TASK-10 — Matches

Modelo: Match

Método	Rota	Auth
GET	/phases/:phaseId/matches?status=&round=	Público
GET	/matches/:id	Público
POST	/phases/:phaseId/matches	DISCIPLINE_MANAGER
PATCH	/matches/:id	DISCIPLINE_MANAGER
PATCH	/matches/:id/status	DISCIPLINE_MANAGER

POST /phases/:phaseId/matches

// request

```
{
  "groupId": "clx_group_a",
  "round": 1,
  "bracketSlot": null,
  "entryAId": "clx_entry_cinfc",
  "entryBId": "clx_entry_ctgfc",
  "scheduledAt": "2026-10-13T14:00:00Z",
  "venue": "Ginásio CIn"
}
```

`entryBId: null` representa um bye (mata-mata com número ímpar de entradas).

GET /matches/:id

```
{
  "data": {
    "id": "clx_match_01",
    "phaseId": "clx_phase_grupos",
    "groupId": "clx_group_a",
    "round": 1,
    "entryA": { "id": "clx_entry_cinfc", "name": "CIn FC" },
    "entryB": { "id": "clx_entry_ctgfc", "name": "CTG United" },
    "scoreA": 2,
    "scoreB": 1,
    "status": "LIVE",
    "scheduledAt": "2026-10-13T14:00:00Z",
    "venue": "Ginásio CIn",
    "lastEventSequence": 7
  }
}
```

Nota crítica para o frontend: `scoreA/scoreB` são a fonte de verdade para exibir placar — não recalcule a partir de `MatchEvent` no cliente. `lastEventSequence` indica quantos eventos já existem; use isso para saber se o feed de eventos que você tem em cache está desatualizado.

PATCH /matches/:id/status

```
{ "status": "LIVE" }
```

`status` ∈ `SCHEDULED` | `LIVE` | `FINISHED` | `WALKOVER` | `CANCELLED` | `POSTPONED`. Transição para `FINISHED` dispara recálculo de `PhaseStanding` no backend (TASK-13) — não é responsabilidade do frontend.

TASK-11 — Match Events

Modelo: `MatchEvent`

Método	Rota	Auth
GET	<code>/matches/:matchId/events</code>	Público
POST	<code>/matches/:matchId/events</code>	<code>DISCIPLINE_MANAGER</code>
DELETE	<code>/matches/:matchId/events/:id</code>	<code>DISCIPLINE_MANAGER</code> (correção de erro)

`POST /matches/:matchId/events`

// request — gol de futsal

```
{  
  
  "entryId": "clx_entry_cinfc",  
  
  "athleteId": "clx_athlete_09",  
  
  "type": "GOAL",  
  
  "metadata": { "minute": 34 }  
}
```

// request — cartão

```
{  
  
  "entryId": "clx_entry_ctgfc",  
  
  "athleteId": "clx_athlete_22",  

```

```
"type": "YELLOW_CARD",  
  
"metadata": { "minute": 51 }  
  
}
```

```
// request — set de vôlei
```

```
{  
  
"entryId": "clx_entry_cinfc",  
  
"type": "SET_WON",  
  
"metadata": { "setNumber": 3, "pointsHome": 25, "pointsAway": 20 }  
  
}
```

`type` ∈ GOAL | ASSIST | YELLOW_CARD | RED_CARD | POINT | SET_WON | FOUL
| TIMEOUT_CALLED | SUBSTITUTION | DISQUALIFICATION | CHECKMATE |
WALKOVER_DECLARED | OTHER. `metadata` varia por `type` e por modalidade — não é
validado por schema fixo, apenas documentado por convenção (ver Apêndice B).

```
// response 201
```

```
{  
  
"data": {  
  
"id": "clx_event_08",  
  
"matchId": "clx_match_01",  
  
"type": "GOAL",  
  
"sequence": 8,  
  
"occurredAt": "2026-10-13T14:34:12Z",  
  
"metadata": { "minute": 34 }  
  
}
```

```
}
```

O backend, na mesma transação, incrementa `Match.lastEventSequence`, atualiza `Match.scoreA/scoreB` se `type` afetar placar, e publica o evento no canal de real-time (TASK-12). O frontend não precisa (e não deve) fazer esse cálculo.

GET /matches/:matchId/events

Retorna a lista ordenada por `sequence` (não por `occurredAt`):

```
{
  "data": [
    { "id": "clx_event_01", "type": "GOAL", "sequence": 1, "metadata": { "minute": 5 } },
    { "id": "clx_event_02", "type": "YELLOW_CARD", "sequence": 2, "metadata": { "minute": 12 } }
  ]
}
```

TASK-12 — Real-time (Server-Sent Events)

Método	Rota	Auth
GET	/matches/:matchId/stream	Público

Content-Type: `text/event-stream`. O frontend consome via `EventSource`:

```
const es = new EventSource(`/api/v1/matches/${matchId}/stream`);

es.addEventListener("match-event", (e) => {

  const payload = JSON.parse(e.data);

  // payload: { type, sequence, entryId, athleteId, metadata, scoreA, scoreB }

});
```

Formato de cada mensagem enviada pelo servidor:

event: match-event

id: 8

data:

```
{"type":"GOAL","sequence":8,"entryId":"clx_entry_cinfc","athleteId":"clx_athlete_09","metadata":{"minute":34},"scoreA":2,"scoreB":1}
```

Notas para o mock do frontend:

- Usar o **id** da mensagem SSE (igual a **sequence**) para suportar reconexão automática do **EventSource** via **Last-Event-ID** — se a conexão cair, o servidor deve reenviar a partir desse ponto (implementação real lê o Redis stream/replay a partir da sequência; no mock, simule reenvio dos eventos faltantes).
- Emitir um evento **heartbeat** a cada ~25s (comentário SSE : **ping\n\n**) para manter proxies intermediários de não fecharem a conexão por timeout de inatividade — o frontend deve ignorá-lo.
- No mock local (antes do backend estar pronto), simule com um pequeno servidor Express/Next API route que emite eventos de exemplo a cada poucos segundos, usando o mesmo formato acima.

TASK-13 — Phase Standings (classificação)

Modelo: **PhaseStanding**

Método	Rota	Auth
GET	/phases/:phaseId/standings	Público

```
{  
  
  "data": [  
  
    {  
  
      "entryId": "clx_entry_cinfc",
```



```
"entryName": "Cln FC",

"played": 3, "won": 2, "drawn": 1, "lost": 0,

"scoreFor": 8, "scoreAgainst": 3,

"points": 7, "rank": 1

},

{

"entryId": "clx_entry_ctgfc",

"entryName": "CTG United",

"played": 3, "won": 1, "drawn": 1, "lost": 1,

"scoreFor": 5, "scoreAgainst": 4,

"points": 4, "rank": 2

}

]

}
```

Já vem ordenado por **rank**. Recalculado pelo backend a cada **Match** que entra em **FINISHED** — o frontend só lê.

TASK-14 — Audit Logs

Modelo: **AuditLog**

Método	Rota	Auth
GET	/editions/:editionId/audit-logs?entityType=&staffId=&from=&to=	EDITION_ADMIN
GET	/audit-logs?entityType=&staffId=&from=&to=	Admin (global, sem editionId)

```
{
  "data": [
    {
      "id": "clx_audit_01",
      "staffName": "Ana Coordenadora",
      "action": "MATCH_SCORE_UPDATED",
      "entityType": "Match",
      "entityId": "clx_match_01",
      "beforeData": { "scoreA": 1, "scoreB": 1 },
      "afterData": { "scoreA": 2, "scoreB": 1 },
      "createdAt": "2026-10-13T14:34:12Z"
    }
  ],
  "meta": { "page": 1, "pageSize": 20, "total": 340, "totalPages": 17 }
}
```

TASK-15 — Rotas públicas agregadas (spectator)

Endpoints de conveniência, otimizados para as telas mais acessadas pelo público — evitam N chamadas encadeadas no frontend.

Método	Rota	Descrição
GET	<code>/editions/:editionId/live</code>	Todas as partidas com <code>status=LIVE</code> na edição, com placar atual
GET	<code>/editions/:editionId/schedule?date=</code>	Agenda de partidas do dia
GET	<code>/tournaments/:id/bracket</code>	Estrutura de chaveamento completa (fases KNOCKOUT), pronta para render de árvore

`GET /editions/:editionId/live`

```
{
  "data": [
    {
      "matchId": "clx_match_01",
      "tournamentName": "Futsal Masculino",
      "disciplineName": "Futsal",
      "entryA": "Cln FC", "entryB": "CTG United",
      "scoreA": 2, "scoreB": 1,
      "venue": "Ginásio Cln"
    }
  ]
}
```

```
}
```

Esta é provavelmente a rota mais chamada do sistema no dia do evento — é a que popula a tela "Ao Vivo Agora". Considere que o frontend some com polling leve (ex: a cada 30s) SOMENTE para a lista de partidas ao vivo em si (quais partidas estão rolando); o placar de cada partida individual vem do SSE (TASK-12) depois que o usuário entra na tela daquela partida.

```
GET /tournaments/:id/bracket
```

```
{
```

```
  "data": {
```

```
    "format": "GROUP_KNOCKOUT",
```

```
    "phases": [
```

```
      {
```

```
        "phaseId": "clx_phase_grupos", "name": "Fase de Grupos", "type": "GROUP",
```

```
        "groups": [
```

```
          { "name": "Grupo A", "standings": [ /* mesmo shape da TASK-13 */ ] }
```

```
        ]
```

```
      },
```

```
      {
```

```
        "phaseId": "clx_phase_mata", "name": "Mata-mata", "type": "KNOCKOUT",
```

```
        "matches": [
```

```
          { "round": 1, "bracketSlot": 1, "entryA": "CIn FC", "entryB": "CTG United", "scoreA": 2, "scoreB": 1, "winner": "CIn FC" },
```

```
          { "round": 1, "bracketSlot": 2, "entryA": "Poli Team", "entryB": null, "winner": "Poli Team" }
```

```
        ]
```

$$\left. \begin{array}{l} \} \\] \\ \} \\ \} \end{array} \right\}$$

`entryB: null + winner` preenchido = bye avançando automaticamente; o componente de árvore do frontend deve tratar esse caso sem quebrar o layout.

Apêndice A — Enums de referência (para popular selects/mocks)

EditionStatus: PLANNING | ONGOING | FINISHED | ARCHIVED

TournamentFormat: SINGLE_ELIMINATION | GROUP_KNOCKOUT | LEAGUE_KNOCKOUT

| LEAGUE_ONLY | LEAGUE_LIMITED_KNOCKOUT

TournamentStatus: DRAFT | SCHEDULED | ONGOING | FINISHED | CANCELLED

PhaseType: GROUP | LEAGUE | KNOCKOUT

MatchStatus: SCHEDULED | LIVE | FINISHED | WALKOVER | CANCELLED | POSTPONED

RosterStatus: ACTIVE | INJURED | SUSPENDED | WITHDRAWN

EditionStaffRoleType: EDITION_ADMIN | DISCIPLINE_MANAGER

EventType: GOAL | ASSIST | YELLOW_CARD | RED_CARD | POINT | SET_WON

| FOUL | TIMEOUT_CALLED | SUBSTITUTION | DISQUALIFICATION |

| CHECKMATE | WALKOVER | DECLARED | OTHER

Apêndice B — Convenção de **metadata** por modalidade (MatchEvent)

Não é validado por schema de banco (é JSONB), mas o backend deve validar por DTO específico por combinação **discipline + type** antes de persistir. Tabela de referência para o frontend montar os formulários corretos:

Modalidade	type comum	metadata esperado
Futsal, Handebol	GOAL	{ "minute": number }
Futsal, Handebol	YELLOW_CARD / RED_CARD	{ "minute": number }
Vôlei	SET_WON	{ "setNumber": number, "pointsHome": number, "pointsAway": number }
Basquete	POINT	{ "points": 1 2 3, "quarter": number }
Tênis de mesa	SET_WON	{ "setNumber": number, "pointsHome": number, "pointsAway": number }
Natação	OTHER	{ "timeSeconds": number, "lane": number }
Xadrez	CHECKMATE / WALKOVER_DECLARED	{ "movesCount"?: number }

Apêndice C — Ordem recomendada de implementação/mock

Para o time de frontend começar a montar telas com mocks antes do backend existir, esta é a ordem de valor: **TASK-15 (live/schedule) e TASK-12 (SSE)** são as telas mais visíveis no dia do evento — vale mockar primeiro com dados estáticos girando em loop, mesmo antes de TASK-01 a 11 estarem implementadas de verdade. As telas administrativas (TASK-02 a 09) podem ser mockadas em paralelo usando os exemplos de request/response deste documento diretamente como fixtures (MSW, JSON Server, ou similar).